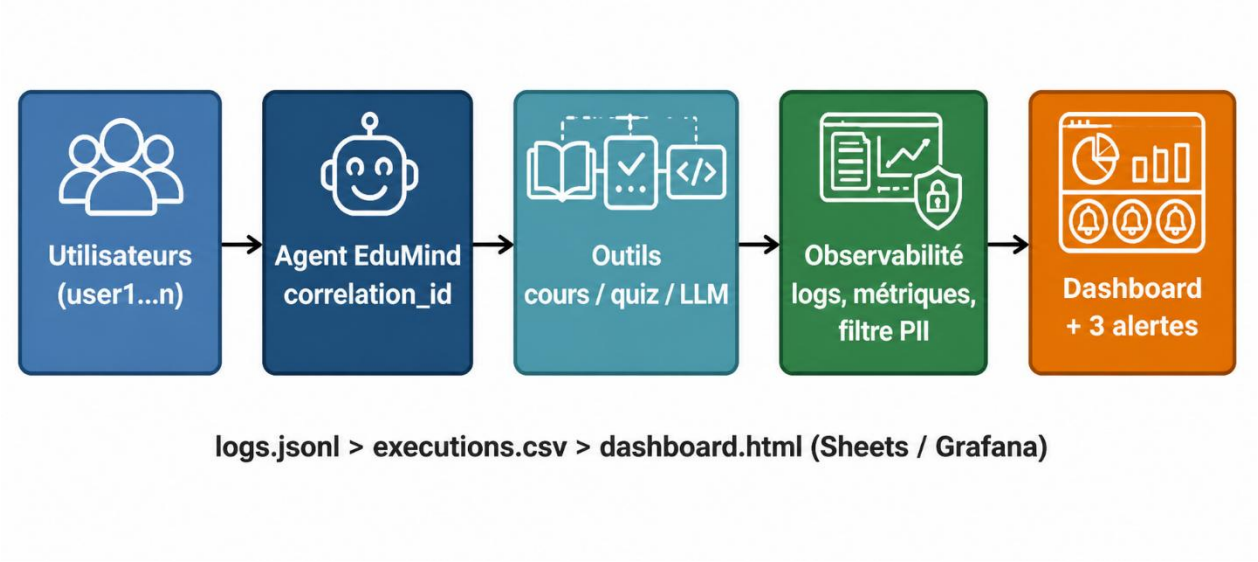
AI in Education

Observabilité de l'agent EduMind - Atelier pratique, Jour 21 – KAMAL EL AFOUI

1. Objectif

Instrumenter l'agent pédagogique EduMind pour collecter logs, métriques et alertes, puis les visualiser dans un dashboard de supervision afin de suivre son activité en temps réel.

2. Architecture



Chaque requête reçoit un correlation_id unique (REQ-xxxxxx) propagé à toutes les étapes : appels LLM, appels d'outils, fin d'exécution. Les trois sorties sont logs.jsonl (événements), executions.csv (une ligne par requête) et dashboard.html (supervision).

3. Les trois piliers de l'observabilité

- Logs : appels d'outils, durée de chaque étape, tokens et coût par appel LLM (logs.jsonl).
- Métriques : taux de succès, latence moyenne et p95, coût total et moyen par requête.
- Traces : reconstitution du parcours complet d'une requête via son correlation_id (onglet Traçabilité du dashboard).

4. Protection des données (PII)

Le filtre est appliqué avant toute écriture dans les logs ; le notebook vérifie leur absence.

Donnée sensible	Exemple	Remplacée par
Email	eleve@ecole.ma	[EMAIL]
Carte bancaire	16 chiffres	[CARD]
Mot de passe / clé d'API	password: abc123	password=[REDACTED]
Téléphone	+212 6 12 34 56 78	[PHONE]

5. Alertes documentées

Alerte	Seuil	Action corrective
Latence	Moyenne > 5 s	Identifier l'outil ou le LLM lent, réduire le contexte.
Taux d'échec	> 20 % des exécutions	Filtrer les logs par correlation_id, corriger l'outil fautif.
Coût	Cumul > 0,05 € (budget cumulé)	Détecter une boucle ou un prompt trop long, plafonner les tokens.

Les trois alertes sont évaluées par la fonction alerts() de main.py et affichées dans le dashboard (vue d'ensemble et onglet Alertes). Le notebook en démontre le déclenchement sur des jeux de données simulés.

6. Résultats de la validation (40 requêtes simulées)

Indicateur	Valeur mesurée	Alerte
Taux de succès	85 % (34/40)	Taux d'échec 15 % : pas d'alerte (seuil 20 %)
Latence moyenne	≈ 0,13 s	Pas d'alerte (seuil 5 s)
Coût cumulé	0,3977 €	ALERTE coût (budget 0,05 €)
Coût moyen par requête	≈ 0,0099 €	-
Échecs	6 timeouts course_db	Traçables par correlation_id

L'alerte coût se déclenche car le budget de 0,05 € est un budget cumulé volontairement bas pour la démonstration ; en production on l'ajuste à la période et au volume suivis.

7. Workflow Langflow (export JSON)

Le flux Langflow Agent Radar est fourni dans EdTech_kamal_2026_corrige.json (dossier flows/) : 10 composants (Chat Input, prompts, interpréteurs Python, adaptateur LLM OpenRouter, requête MongoDB, Chat Output) avec idempotence, 3 tentatives (retry), file de rejet (DLQ) et supervision. Aucun secret n'est exporté : définir les variables OPENROUTER_API_KEY et MONGODB_URI avant exécution.

8. Livrables

Fichier	Rôle
main.py	Agent instrumenté : génère logs.jsonl, executions.csv et met à jour dashboard.html.
main_langgraph.py	Même agent sous forme de graphe LangGraph (pip install langgraph).
executions.csv	Table d'exécutions avec correlation_id.
logs.jsonl	Journal d'événements (start, llm_call, tool_call, end), PII masquée.
dashboard.html	KPI, graphiques, table d'exécutions, alertes, traçabilité, formulaire de test.
notebook.ipynb	Validation : unicité des IDs, PII, trace, 3 alertes.
EdTech_kamal_2026_corrige.json	Workflow Langflow final.
documentation.docx	Documentation du workflow final.
presentation.pptx	Support de présentation de l'atelier.

9. Exécution

- python main.py (ou python main_langgraph.py) puis ouvrir dashboard.html dans un navigateur.
- jupyter notebook notebook.ipynb pour rejouer la validation.
- Dépôt GitHub : voir README.md pour les commandes git.

Remarque : exécuter un seul des deux scripts à la fois, car ils écrivent les mêmes fichiers de sortie.